

MACHINE LEARNING IN FINANCE

MODULE 5 | LESSON 2

SUPPORT VECTOR MACHINES IN PRACTICE

Reading Time 20 minutes

Prior Knowledge Classification, Support Vector Machines, Decision Trees.

Keywords Hyperparameters, slack variable, returns, Receiver Operating Curve (ROC).

In the previous lesson, we covered the methodology of Support Vector Machines. In this lesson, we apply it to a predictive analysis problem.

1. Introduction

In this lesson, we will implement SVM on a trading strategy for the Luxembourg index (LUXXX) based on other country indices as well as technical indicators. The strategy is to take a long/short position when a 0.25% change in LUXXX return is predicted. The long/short position will be based on an upward or downward trending market. Keep in mind that this strategy does not take into account trading costs, but the lesson is designed to show the practicality of SVM in predictive analytics for financial problems.

We will begin by importing the necessary packages.

```
In [ ]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn import svm
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.tree import DecisionTreeClassifier
```

The data containing the weekly closing price of indices can be found in the following csv file along with the dates. We load this data and thereafter view it using the head() command.

Should you store the csv file in the same location as the notebook, then use the uncommented cells below. Use the commented cells instead if you choose to store the csv in a different location. The "loc" variable is the path to the csv you need to specify.

```
In [ ]: # Loc = "ENTER YOUR FULL PATH TO LOCATION OF DATA FILE HERE"
# data_df = pd.read_csv(Loc+"/MScFE 650 MLF GWP Data.csv")
data_df = pd.read_csv("MScFE 650 MLF GWP Data.csv")
data_df.head(3)
```

The 'Date' column is in a string format; therefore, we convert it to a datetime format.

```
In [ ]: # Convert string to datetime
data_df["Date"] = pd.to_datetime(data_df["Date"])
```

Below is where we specify the target column. In our case, it is the LUXXX index. The returns of LUXXX need to be calculated since our strategy is based on the returns over the weeks. The remaining indices are converted to returns instead of their prices. This accomplishes two things, namely it scales the features into a common unit as well as captures movement in the prices instead.

```
In [ ]: # Set Target Index for predicting
target ETF = "LUXXX"

# Use returns instead of prices for other Indices
# Other Indices used as Index_features
ETF_features = data_df.loc[:, ~data_df.columns.isin(["Date", target ETF])]
data_df[ETF_features] = data_df[ETF_features].pct_change()

data_df[target ETF + "_returns"] = data_df[target ETF].pct_change()
```

The cell below calculates the Target column and converts it into a categorical binary variable. We denote 1 for the absolute returns of LUXXX exceeding 0.25% and 0 otherwise. In a downward or bear market, we look to short should the price decrease by more than this threshold and we take a long position in a bear market. The goal here is to predict a percent change more than a threshold. Note that we shift the target column by one week to align the predicted period with the period available for other predictors. The ML algorithm requires the label and feature/predictor values to be captured in the same row or observation.

```
In [ ]: # Create Target Column.
# Shift period for target column
data_df[target ETF + "_returns" + "_Shift"] = data_df[target ETF + "_returns"].shift(1)

# Strategy to take long position for anticipated returns of 0.5%
data_df["Target"] = np.where(
    (data_df[target ETF + "_returns_Shift"].abs() > 0.025), 1, 0
)
```

It is worth looking at any imbalance in the dataset. The proportion of 34.5% of the target class 1 does not indicate any imbalance.

```
In [ ]: # Checking target proportion
round(data_df["Target"].sum() / len(data_df), 4)
```

Now that we have our Target class, we can select which indices contribute to predicting the target class. We use the feature importance tool in `Sklearn` for a Decision Tree to

do this. We look at the cumulative importance for the features that make a contribution > 0.

```
In [ ]: # Preparing for Train/Test split

NoNaN_df = data_df.dropna()
X = NoNaN_df[ETF_features]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_

# USE DECISION TREE FOR FEATURE IMPORTANCE
# Fitting Decision Tree Classification to the Training set

DTree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0,
    max_depth=8,
    min_samples_leaf=30,
    min_samples_split=20,
    # , class_weight=classweight
)
DTree.fit(X_train, y_train)

feature_importances = pd.DataFrame(
    DTree.feature_importances_,
    index=X.iloc[:, :].columns, # don't want the last Target column
    columns=["importance"],
).sort_values("importance", ascending=False)

feature_importances["Cumul_Imp"] = feature_importances.cumsum().iloc[:, :]
# If you want to see only those that explain a % of variation
pct_var = 1.0
feature_importances = feature_importances[(feature_importances["Cumul_Imp"] <= p
```

The cumulative importance shows only 4 indices that contribute, namely MSCI KOREA, DENMARK, FRANCE, and NORWAY. This reduces the features significantly and will also reduce the run time when doing hyperparameter tuning.

```
In [ ]: # PLOT IMPORTANCE

# Example data
cols = feature_importances.index
y_pos = np.arange(len(cols))
performance = feature_importances.importance

df2plot = pd.DataFrame(data=performance, index=feature_importances.index)
df2plot["Variable"] = feature_importances.index
df2plot = df2plot[df2plot["importance"] > 0.000]

print(df2plot)
```

```
f, ax = plt.subplots(figsize=(6, 5))

sns.barplot(
    x="importance", y="Variable", data=df2plot, label="Variable Ranking", color=
)

ax.set(xlim=(0, 1), ylabel="", xlabel="importance")

for p in ax.patches:
    width = p.get_width()
    ax.text(
        width + 0.05,
        p.get_y() + p.get_height() / 2.0 + 0.2,
        "{:1.2f}".format(width),
        ha="center",
    )
```

Figure 1: Feature Importance Contribution of Four Predictors

```
In [ ]: # Indices to add value in prediction
ETF_ImpFeatures = []
for i in df2plot["Variable"].values:
    ETF_ImpFeatures.append(i)

ETF_ImpFeatures
```

2. Create Technical Indicators

Among the first technical indicators are the moving average crossovers of short- (SMA_5) and long-term (SMA_15). We use the ratio of (SMA_15/SMA_5) to capture the relationship as one feature and any crossovers. We will rely on the SVM model to capture the relationship between this feature and the target class.

```
In [ ]: data_df["SMA_5"] = data_df[target ETF].rolling(5).mean()
data_df["SMA_15"] = data_df[target ETF].rolling(15).mean()
data_df["SMA_ratio"] = data_df["SMA_15"] / data_df["SMA_5"]

# Can drop SMA columns since not needed anymore.
data_df.drop(["SMA_5", "SMA_15"], axis=1, inplace=True)
```

The next technical indicator we look at is the relative strength index (RSI), which is a momentum oscillator. This indicator measures the magnitude of recent price changes. The target price is usually considered overbought when the RSI > 70% and oversold when < 30%. This is based on the price change of LUXXX before the week of prediction.

```
In [ ]: # shift the price of the target by 1 unit previous in time
data_df["Diff"] = data_df[target ETF] - data_df[target ETF].shift(1)
data_df["Up"] = data_df["Diff"]
data_df.loc[(data_df["Up"] < 0), "Up"] = 0

data_df["Down"] = data_df["Diff"]
data_df.loc[(data_df["Down"] > 0), "Down"] = 0
data_df["Down"] = abs(data_df["Down"])

data_df["avg_5up"] = data_df["Up"].rolling(5).mean()
```

```

data_df["avg_5down"] = data_df["Down"].rolling(5).mean()

data_df["avg_15up"] = data_df["Up"].rolling(15).mean()
data_df["avg_15down"] = data_df["Down"].rolling(15).mean()

data_df["RS_5"] = data_df["avg_5up"] / data_df["avg_5down"]
data_df["RS_15"] = data_df["avg_15up"] / data_df["avg_15down"]

data_df["RSI_5"] = 100 - (100 / (1 + data_df["RS_5"]))
data_df["RSI_15"] = 100 - (100 / (1 + data_df["RS_15"]))

data_df["RSI_ratio"] = data_df["RSI_5"] / data_df["RSI_15"]

# Can drop RS Calc columns columns
data_df.drop(
    ["Diff", "Up", "Down", "avg_5up", "avg_5down", "avg_15up", "avg_15down"],
    axis=1,
    inplace=True,
)

```

Rate of change is the percentage change in price over a 15-week period. It is a long-term rate of change and is also a momentum indicator. It is also used to spot overbought and oversold instances.

```
In [ ]: data_df["RC"] = data_df[target ETF].pct_change( periods=15)
```

We now combine all features, that is the indices' price change as well as the technical indicators.

```
In [ ]: # all_feats
ETF_ImpFeatures.append("SMA_ratio")
ETF_ImpFeatures.append("RSI_ratio")
ETF_ImpFeatures.append("RC")
```

We are now in a position to train our model. To begin, we perform the train/test split of the data into an 80/20 split.

```
In [ ]: # Train/Test split
# Train/Test split. No NaNs in the data.
NoNaN_df = data_df.dropna()
X = NoNaN_df[ETF_ImpFeatures]

X = X.iloc[:, :] # .values
y = NoNaN_df.loc[:, "Target"] # .values

del NoNaN_df

# from sklearn.cross_validation import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

To determine the best kernel and slack variable value, we do a grid search method using `GridSearchCV`. This will explore the specified hyperparameter space for the optimal values over five cross validations.

```
In [ ]: # defining parameter range
param_grid = {
```

```

    "C": [0.01, 0.1, 1, 10, 100, 1000],
    # 'gamma': [1, 0.1, 0.01, 0.001, 0.0001],
    "kernel": ["rbf", "linear", "poly"],
}

grid = GridSearchCV(svm.SVC(), param_grid, refit=True, verbose=3, cv=5)

# fitting the model for grid search
grid.fit(X_train, y_train)

```

The best hyperparameters are specified below. We can see that the best kernel to use for this dataset is the polynomial kernel with a slack variable value of 10.

```

In [ ]: # print best parameter after tuning
print(grid.best_params_)

# print how our model looks after hyper-parameter tuning
print(grid.best_estimator_)

```

Finally, we can train our SVM model for the specified hyperparameters and compare it to an untuned decision tree classifier.

```

In [ ]: # Train with Tuned SVM
# Create a svm Classifier
clf_tuned = svm.SVC(
    random_state=0,
    C=grid.best_params_["C"],
    kernel=grid.best_params_["kernel"],
    probability=True,
)

clf_tuned.fit(X_train, y_train)

# Fitting Decision Tree Classification to the Training set as well

clf_tree = DecisionTreeClassifier(
    criterion="entropy",
    random_state=0,
    max_depth=8,
    min_samples_leaf=30, # DM
    min_samples_split=20,
    # , class_weight=classweight
) # DM
clf_tree.fit(X_train, y_train)

```

From the results below, we see that the SVM model is significantly better than a random guess; however, the tree-based approach performed slightly better with a 4% increase in AUC.

3. Performance

```

In [ ]: # Performance

# predicted probabilities generated by sklearn classifier

```

```

y_pred_proba = clf_tuned.predict_proba(X_test)
y_pred_probatree = clf_tree.predict_proba(X_test)

# SVM ROC dependencies
fpr, tpr, _ = roc_curve(y_test, y_pred_proba[:, 1])
# TREE ROC dependencies
fpr_tree, tpr_tree, _ = roc_curve(y_test, y_pred_probatree[:, 1])

auc = round(roc_auc_score(y_test, y_pred_proba[:, 1]), 4)
auc_tree = round(roc_auc_score(y_test, y_pred_probatree[:, 1]), 4)

# SVM Model
plt.plot(fpr, tpr, label="SVM, auc=" + str(auc))
# Tree model
plt.plot(fpr_tree, tpr_tree, label="Tree, auc=" + str(auc_tree))
# Random guess model
plt.plot(fpr, fpr, "-", label="Random")
plt.title("ROC")
plt.ylabel("TPR")
plt.xlabel("FPR")

plt.legend(loc=4)
plt.show()

```

Figure 2. ROC Curve for Three Models

4. Conclusion

This lesson focused on implementing the SVM model in Python on a predictive analytical problem, specifically a trading strategy for long positions of an index. We also included technical indicators as part of the predictors. We found that the SVM provides better results than a random guess, and the results are not too far off from those of a decision tree classifier. As an exercise, you can adjust this notebook to look at a hyperparameter-tuned decision tree classifier as well. In the next lesson, we will look at a model that is very effective on non-linear data, namely an artificial neural network.